

Reinforcement Learning in Game Applications

By Hugo, Vamsi, Theophanes



What is Q-Learning?

- Q-Learning is a type of reinforcement learning. (no data but has model of environment)
- What is an appropriate scenario to use it?
- Using incentivized value placements, we can train a model to "want" to take more effective/desirable actions.
- Think of it like a carrot on a stick for this snake.

Bellman's equation

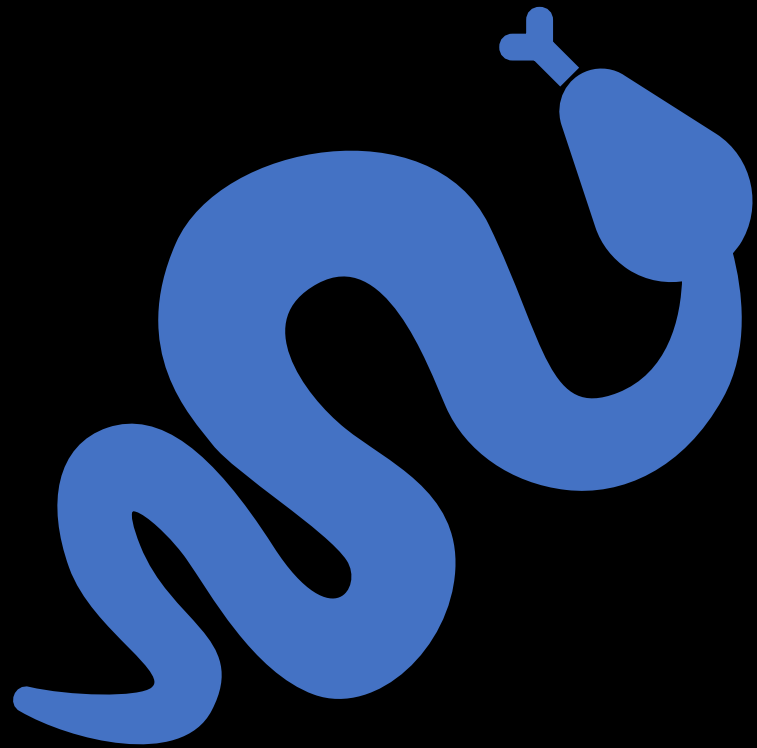
The expected return (value)
at the current state s is:

The expected reward for
taking action a at state s ...

$$V(s) = \max_a (R(s, a) + \gamma V(s'))$$

The maximum value of any
possible action a for:

...plus the discount factor (gamma)
multiplied by the value of the next state



Q-Learning in Snake

A close-up photograph of a soccer ball with black and white hexagonal panels, caught in a light-colored rope goal net. The net is strung over a metal frame, with a thick vertical post visible on the left. The background is a bright blue sky with scattered white clouds. The text "Snake Goal" is overlaid in the upper right area.

Snake Goal

Score of 100



How would this be added
in Snake?

Key Factors

There are many important parts to take account of when using Q-Learning within Snake:

- Map size
- States - Actions
- Reasons for death + rewards
- RANDOMNESS – Learning rate

Initial Implementation

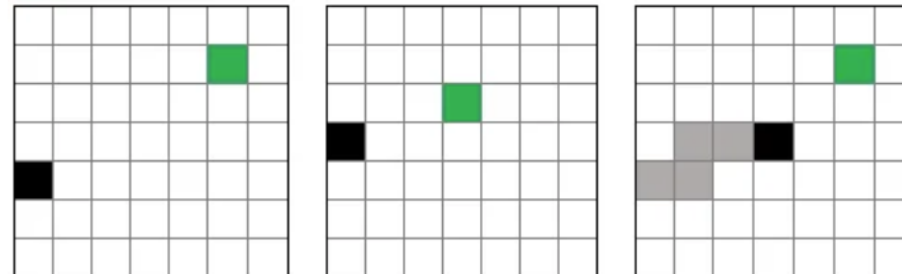
- Rewards
- State actions
- Randomness / epsilon value
- Learning rate +

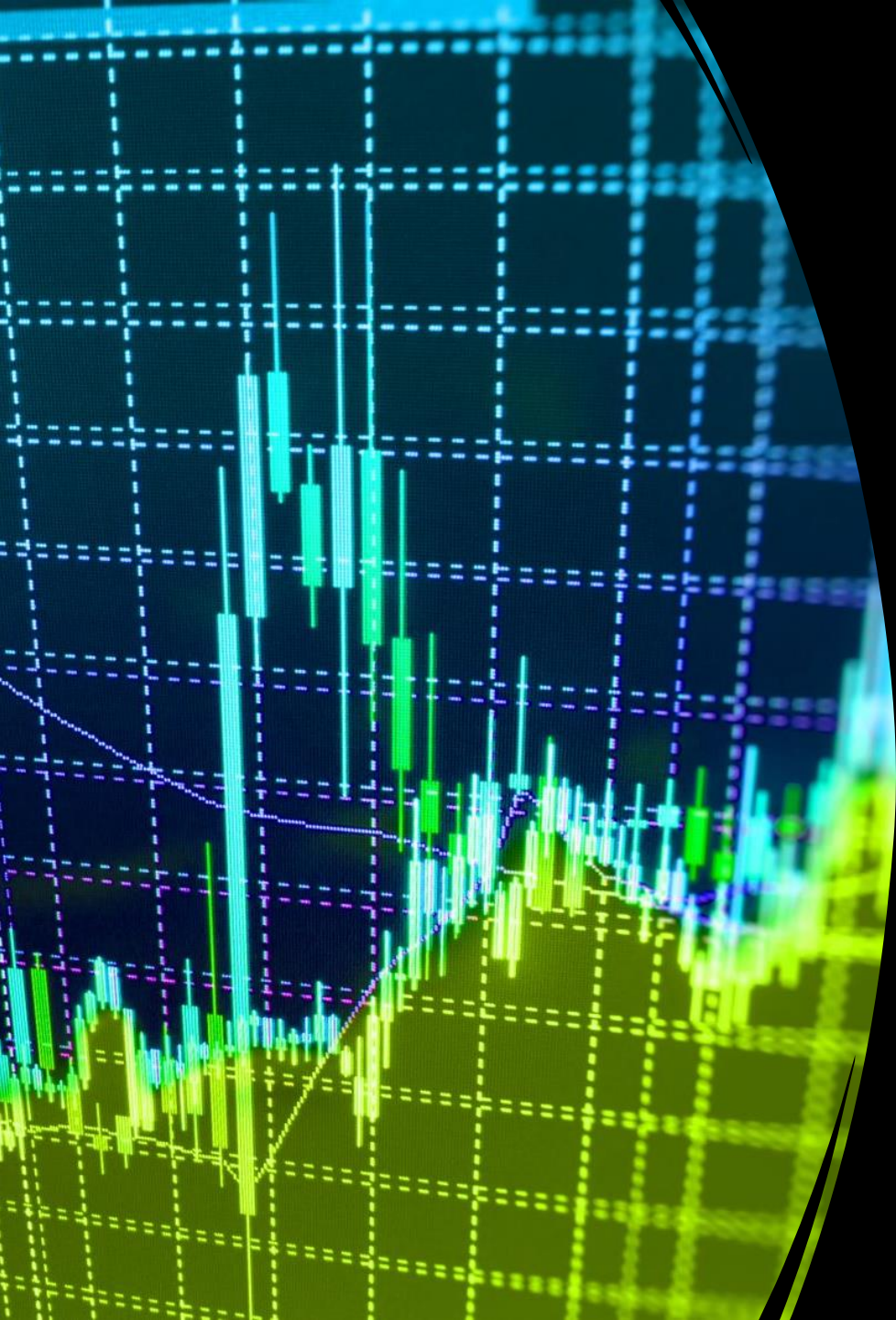
```
# Learning parameters
self.epsilon = 0.1
self.lr = 0.7
self.discount = .5
```

```
if s0.food != s1.food: # Snake ate a food, positive reward
    reward = 1
elif (abs(x1) > abs(x2) or abs(y1) > abs(y2)): # Snake is closer to the food, positive reward
    reward = 0.5
else:
    reward = -0.5 # Snake is further from the food, negative reward
```

■ Snake Head
■ Snake Tail
■ Food

All 3 configurations of the game
are represented by the same
state





Experiments + Changes

Reducing the number of states action

- Should prevent it from going into itself allowing it to automatically not have to learn that
 - Should allow it to learn it better
 - Should make the q values better estimates if there are fewer of them.
-
- RIGHT! But terrible idea
 - Score : 67

Increasing the number of states action

- Less generalization -> more info into decision making -> better actions
- Learns slower (but we have time)

- Helped but not much
- Score: 75

Increasing randomness + different rewards

- Makes it dumb as hell at the beginning
 - Prevents it from getting practice at higher point situations
 - LEARNS TO NOT HIT ITSELF
 - Rewards encourage getting apples not going to them
 - Rewards prevent taking too long
-
- Score : 78

Letting it run for a day and hope it gets lucky

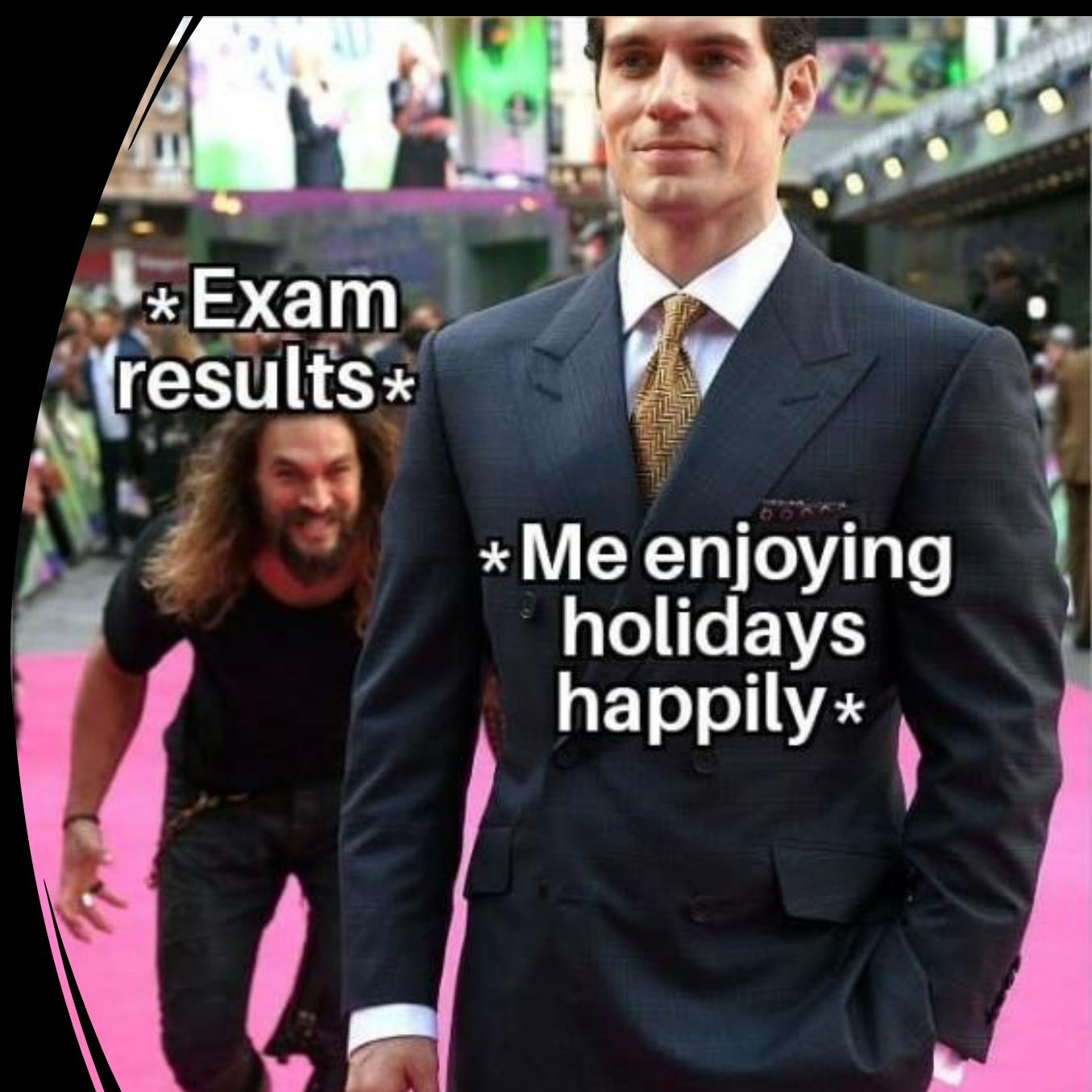
- Good chance of apples being close together
- More learning time -> better QTable
- Randomness being high doesn't limit as much



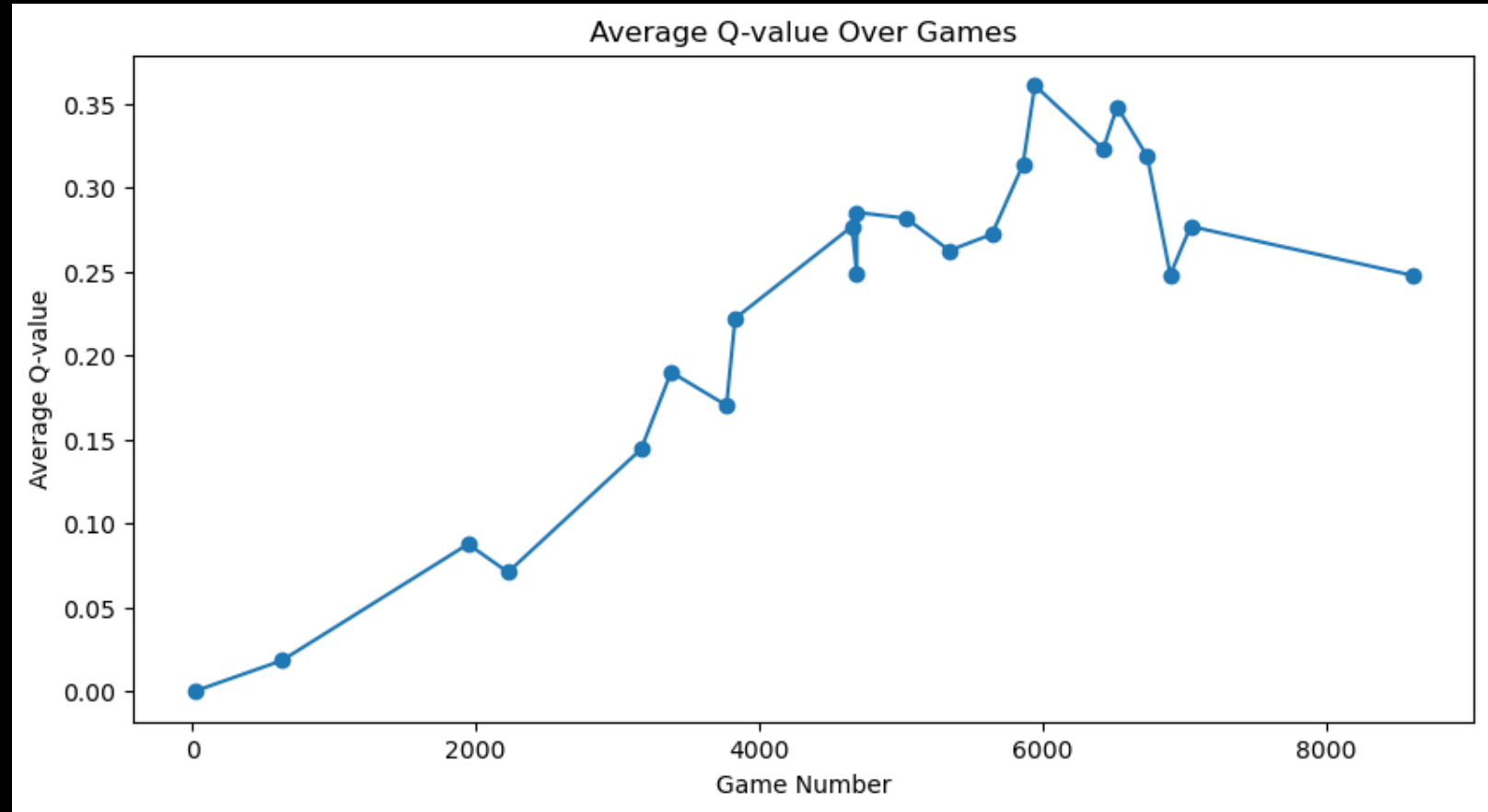
WE DID
IT!!

Score :
108

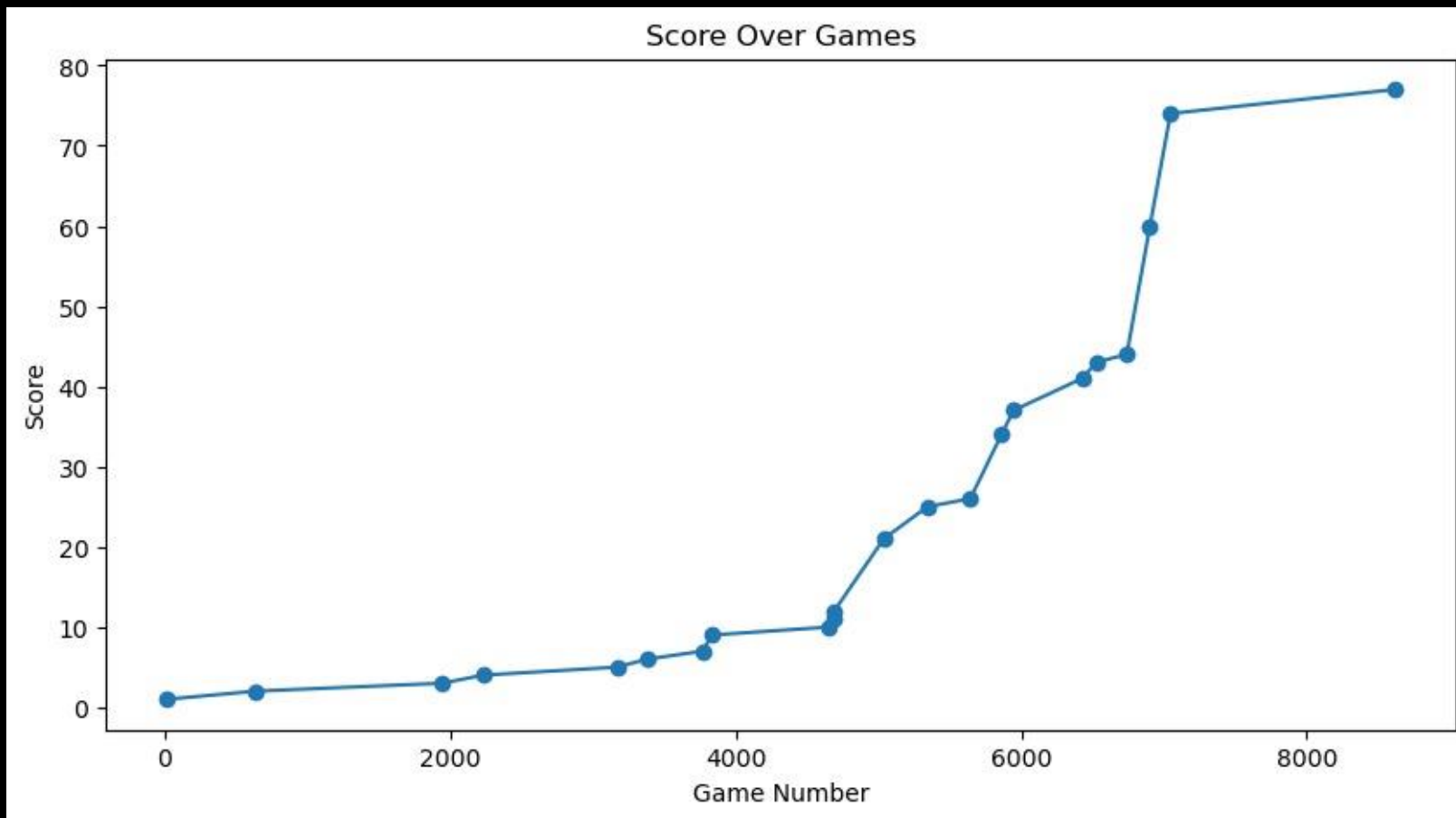
Results



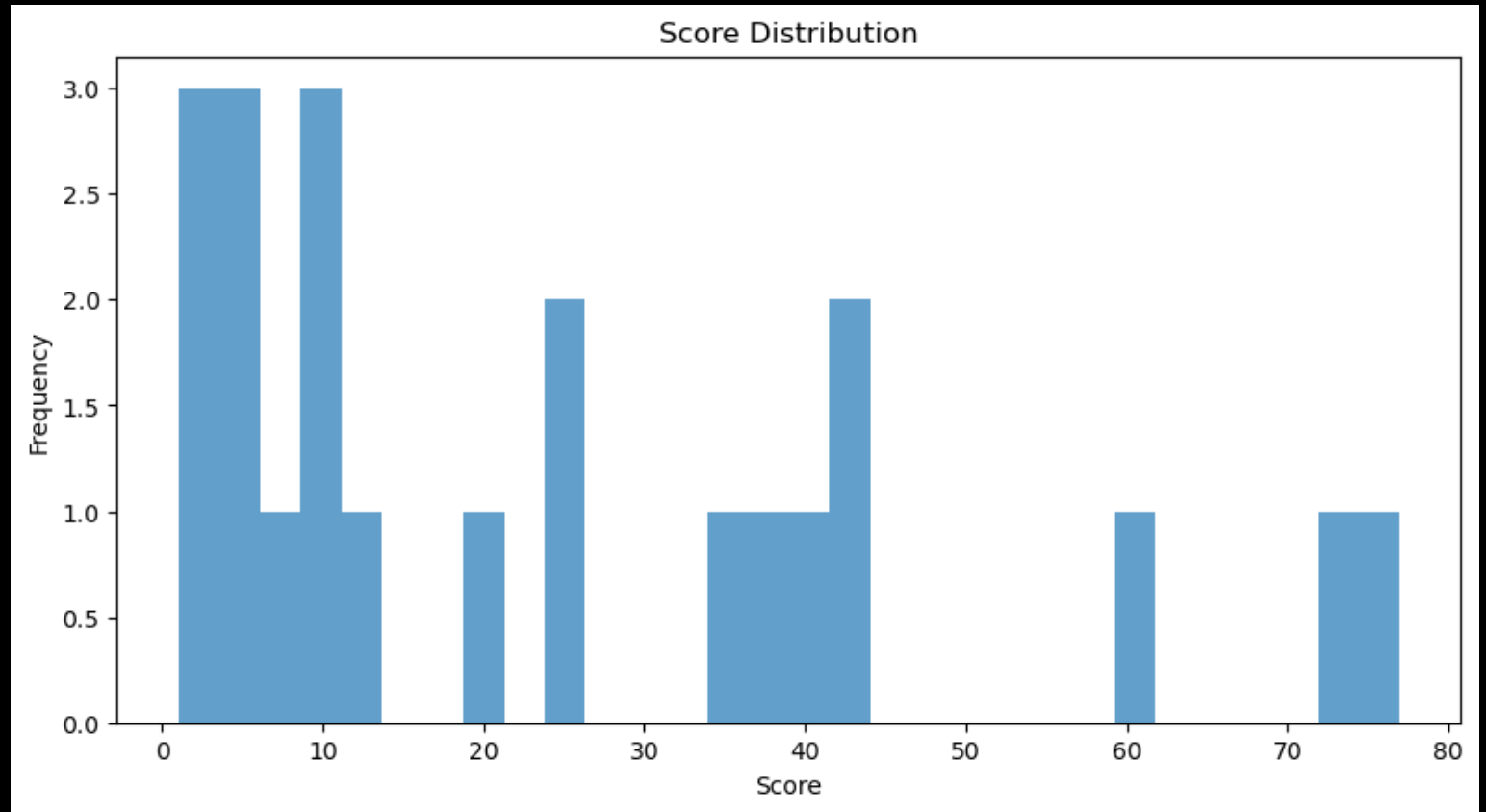
Average Q-value Over Games



Score Over Games



Score Distribution



Future Improvements

- If we had more time, we could fine tune parameters that may have not been the best fit.
- Comparing Q-Learning with other reinforcement-learning models.
- Diversifying the snake behavior further on in its lifeline (e.g snake length > 50 , not dying is more important than score)
- Increase the number states
- DQN – expands possibility.

Citations

- CS2400 Lecture on Qlearning
- Qlearning Jupyter Notebook on the MAIC Website
- CodeBullet on Youtube
- <https://www.geeksforgeeks.org/what-is-reinforcement-learning/>
- <https://github.com/techtribeyt/snake-q-learning/blob/master/snakeql.py>
- <https://towardsdatascience.com/teaching-a-computer-how-to-play-snake-with-q-learning-93d0a316ddc0>

Q + A

- Question and answers